



Figure 1: An OpenSSL change that caused a security vulnerability in Debian

That's HUGE—it's one warning per line of code! In my experience in running such tools on large code bases, I've seen violation logs from such tools exceeding 2 GB in size. That's a humongous amount of data, and we cannot humanly process such a large number of violations.

Again, it depends on the kind of static analyser that you use. For example, all the three tools I've mentioned are 'shallow-analysers' or 'pattern-matchers'. Nevertheless, many widely used static analysers are still such 'pattern-matchers', and hence this discussion is still useful. However, if you run 'deep-analysers' (that use model-checking, abstract interpretation, etc), you'll get a fraction of the number of violations. That brings in another related problem: the cost of the tools. Deep static analysers are mostly commercial tools, and they can find important hard-to-detect problems; but they work out very expensive for regular use in projects.

Another problem with static analysers is that most violations reported by them are not of immediate concern to the projects. For example, assume that you're working on a C project on Lintel (Linux + Intel) and that project is never going to be ported to any other machine or OS. So, when the static analysers report violations related to portability, they are not going to be relevant to you. You will need to 'weed out' irrelevant violations, and look out for pertinent violations—that's irritating, as well as time-consuming.

Dynamic analysers

Now, all is not well with dynamic analysers either. A disadvantage with these is that they need executable code for running the tools, unlike static analysers that can work on partial source code.

However, the main disadvantage with dynamic analysers is that they don't find latent defects in code. In other words, the tools report bugs only if they are in the given code path with the given data values; or else, they will not report the errors. What this means is that you need a great testing infrastructure and high

code coverage to find bugs using dynamic analysers. But look at the contradiction here—if you have a great testing infrastructure, and have high code coverage, it means you would have anyway found a considerable number of bugs with testing!

Also, running dynamic analysers is time-consuming and resource intensive. Further, depending on the kind of dynamic analyser you use, it can change the behaviour of the program! In other words, even if the program has a bug, if you try using a dynamic analyser, it may disappear and may not get reported by the dynamic

analysers! This may seem like strange behaviour to you, but if you've used dynamic analysers or debuggers extensively you'll know that this is a common phenomenon. Why does this happen? Consider the example of concurrency bugs, which depend on thread scheduling. Acts such as monitoring the program, alter thread scheduling such that the exact situations required to reproduce the concurrency bug no longer exist. Hence, dynamic analysers may not find bugs, even when you know that a bug exists in your program!

With these advantages and disadvantages of static and dynamic analysers in mind, you need to take a holistic view in selecting and using these tools for your project. You may even need to change your development process to introduce such tools as part of your development life-cycle. All tools have problems, so you need to figure out smart ways to overcome them. However, if you altogether give up on using program analysers, and depend on testing alone, you can be assured that the software you deliver will be bug-infested!

Static analysers detect most problems that are usually found using manual analysis. Manual analysis is resource-intensive, but very effective; so it makes sense to use the available time of the experts for manual reviews. By mandating that you need to run static analysers and fix all the major problems before code review, manual reviews can focus on finding deeper problems in code. Selecting a suitable static analysis tool for your project is very important. For example, if yours is a development project, it's likely that the code is not stable, and numerous bugs will be introduced as the development progresses. So, by choosing a shallow-analyser, and by enabling selective rules (i.e., disabling 'noisy' rules), you can make best use of the tool. However, if yours is a legacy code-base, it's likely that changes to the code are very controlled. In this situation, you can make the best use of deep static analysers, and find and fix critical bugs in the code.

Similarly, using dynamic analysers as part of the development life-cycle needs considerable thought and

Continued on Page no 99